

Lab 3 – Written Architecture Justification

Alumni Mentorship & Mock Interview Platform

1. Architecture Selection

We chose Layered Architecture for the Alumni Mentorship & Mock Interview Platform. The system is organized into a Presentation Layer (Student & Mentor Portal Component), a Business Layer (Mentor Recommendation Component, Booking Component, Availability Component, Calendar & Meeting Component, and Scorecard Component), and a Data Layer (Data Store & Privacy Component).

2. Scenario-Specific Reasons

First, booking correctness depends on the Booking Component and the Availability Component working together to reject unavailable or already-booked slots and to keep mentor booking records consistent. Keeping both components within one Business Layer, communicating through direct in-process interfaces rather than across independently deployed services, makes it far simpler to guarantee that no mentor ends up with two confirmed sessions in the same slot, which is central to how this platform must behave.

Second, the platform combines several distinct capabilities — recommending mentors by domain, managing availability and bookings, synchronizing calendars and issuing meeting links, and recording mock-interview scorecards. Layered Architecture lets each of these live in its own Business Layer component while the Student & Mentor Portal Component stays a single, stable entry point for students and mentors. The portal does not need to know how recommendations are generated, how slots are validated, or how scorecards are stored, so each business capability can be maintained independently without changing how users interact with the platform.

3. Security Advantage

All user profile data and mock-interview scorecards are held only in the Data Store & Privacy Component, and the Presentation Layer never accesses this data directly — every request must pass through a Business Layer component first. This layering gives the platform one controlled point through which privacy protections and the required 24-hour soft-deletion of profile and scorecard data can be enforced consistently, instead of that logic being scattered across the portal or across multiple independent services.

4. Performance Benefit

Because the Booking Component and Availability Component sit in the same Business Layer and communicate through direct in-process assembly connectors rather than over a network, checking slot availability and confirming a booking involves fewer component hops than a distributed, microservices-style split would require. This keeps booking and calendar-sync operations reasonably responsive without depending on additional network calls between separately deployed services.